

Andy Garcia
Assignment 2
EECS 348
9/15

I compared code from ChatGPT and Claude on google chrome through their websites.
The prompt was copied generated by Gemini using instructions given by the assignment:

Here is a prompt you can feed into a generative AI to create the C program:

****System Prompt / Meta Prompt:****

You are an expert C programmer. Please write a complete, self-contained C program (`main.c`) that implements an email priority system for a CEO based on the following detailed specification:

1. Requirements & Data Structures

* ****Data Types:**** Define an `Email` struct to store `sender` (string), `subject` (string), and `date` (`MM-DD-YYYY` format, parsed into a format suitable for date comparison).

* ****Priority Queue:**** Implement a ****MaxHeap**** from scratch using a dynamic list/array (`Email\* heap`). ****Do not use any external or built-in heap/priority queue libraries.****

* ****Priority Logic:****

1. Primary key: ****Sender Category Priority**** (Higher rank wins):

* `Boss` (Priority 5)

* `Subordinate` (Priority 4)

* `Peer` (Priority 3)

* `ImportantPerson` (Priority 2)

* `OtherPerson` (Priority 1)

2. Secondary key: ****Date**** (Newest email wins if categories match).

* ****Comparison Function:**** Write a `compare(Email a, Email b)` helper function returning standard `-1`, `0`, or `1` values to maintain heap properties consistently during heapify up/down ops.

2. MaxHeap Functions to Implement

* `void initHeap(MaxHeap \*hp)`

* `void insert(MaxHeap \*hp, Email email)`

* `Email peek(MaxHeap \*hp)` (Returns root element without removing)

* `Email extractMax(MaxHeap \*hp)` (Removes and returns top email, re-heapifies down)

* standard `heapifyUp` and `heapifyDown` helper routines.

3. File Input & Commands

Read commands from `stdin` or a test file containing line-by-line commands:

* `EMAIL <sender category>, <subject line>, <date>`

* Parses category, subject line, and `MM-DD-YYYY` date string. Inserts the new email into the MaxHeap.

```

* `NEXT`
* Displays the next email at the top of the heap without removing it:
...

Next email:
Sender: <sender>
Subject: <subject>
Date: <date>

...

* If the queue is empty, handle gracefully (e.g., print a message or do nothing).

* `READ`
* Deletes/pops the highest priority email from the queue without printing its details.
* Must safely handle edge cases like calling `READ` on an empty queue.

* `COUNT`
* Displays the current number of unread emails:
...

There are <count> emails to read.

...

```

4. Code Constraints & Edge Cases

```

* Code must handle consecutive `NEXT` operations without mutating the heap state.
* Code must handle consecutive `READ` operations correctly (removing items without extra output).
* Ensure proper memory allocation and cleanup (free dynamically allocated memory upon program exit).
* Standard C libraries allowed: ``, ``, ``.

```

Write robust, well-commented C code that follows these exact rules.

ChatGPT's Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 10
#define MAX_SENDER 50
#define MAX_SUBJECT 200
#define MAX_DATE 11

/* Represents one email */
typedef struct {
    char sender[MAX_SENDER];
    char subject[MAX_SUBJECT];

```

```

char date[MAX_DATE];

/* Date stored as YYYYMMDD for easy comparison */
int dateValue;

/* Priority based on sender category */
int priority;
} Email;

/* MaxHeap implemented using a dynamic array */
typedef struct {
    Email *heap;
    int size;
    int capacity;
} MaxHeap;

/*
 * Returns the priority associated with a sender category.
 *
 * Higher number = higher priority.
 */
int getPriority(const char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1; /* OtherPerson */
}

/*
 * Converts MM-DD-YYYY into YYYYMMDD.
 *
 * Example:
 * 09-15-2026 -> 20260915
 *
 * This makes comparing dates very simple:

```

```

*    20260915 > 20250820
*/
int parseDate(const char *date)
{
    int month, day, year;

    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3)
        return 0;

    return year * 10000 + month * 100 + day;
}

```

```

/*
* Compares two emails.
*
* Returns:
*    1 if a has higher priority than b
*    0 if they are equal
*   -1 if a has lower priority than b
*
* Primary comparison:
*    Sender category priority
*
* Secondary comparison:
*    Newest date
*/
int compare(Email a, Email b)
{
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)
        return -1;

    /* Same sender priority, so compare dates */
    if (a.dateValue > b.dateValue)
        return 1;

    if (a.dateValue < b.dateValue)
        return -1;

    return 0;
}

```

```

/*
 * Initializes an empty heap.
 */
void initHeap(MaxHeap *hp)
{
    hp->size = 0;
    hp->capacity = INITIAL_CAPACITY;

    hp->heap = malloc(hp->capacity * sizeof(Email));

    if (hp->heap == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
}

/*
 * Swaps two emails.
 */
void swap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/*
 * Moves an element upward until the max-heap property is restored.
 */
void heapifyUp(MaxHeap *hp, int index)
{
    while (index > 0) {
        int parent = (index - 1) / 2;

        /*
         * If the current email is more important than its parent,
         * swap them.
         */
        if (compare(hp->heap[index], hp->heap[parent]) > 0) {
            swap(&hp->heap[index], &hp->heap[parent]);

```

```

        index = parent;
    } else {
        break;
    }
}
}

```

```

/*
 * Moves an element downward until the max-heap property is restored.
 */

```

```

void heapifyDown(MaxHeap *hp, int index)
{
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        /* Check left child */
        if (left < hp->size &&
            compare(hp->heap[left], hp->heap[largest]) > 0) {
            largest = left;
        }

        /* Check right child */
        if (right < hp->size &&
            compare(hp->heap[right], hp->heap[largest]) > 0) {
            largest = right;
        }

        /*
         * If neither child is larger, the heap property
         * has been restored.
         */
        if (largest == index)
            break;

        swap(&hp->heap[index], &hp->heap[largest]);
        index = largest;
    }
}

```

```

/*

```

```

* Inserts an email into the max heap.
*/
void insert(MaxHeap *hp, Email email)
{
    /*
     * If the array is full, double its capacity.
     */
    if (hp->size == hp->capacity) {
        int newCapacity = hp->capacity * 2;

        Email *newHeap = realloc(
            hp->heap,
            newCapacity * sizeof(Email)
        );

        if (newHeap == NULL) {
            fprintf(stderr, "Memory allocation failed.\n");
            free(hp->heap);
            exit(EXIT_FAILURE);
        }

        hp->heap = newHeap;
        hp->capacity = newCapacity;
    }

    /* Put the new email at the end */
    hp->heap[hp->size] = email;

    /* Restore max-heap property */
    heapifyUp(hp, hp->size);

    hp->size++;
}

/*
 * Returns the highest-priority email without removing it.
 */
Email peek(MaxHeap *hp)
{
    return hp->heap[0];
}

```

```

/*
 * Removes and returns the highest-priority email.
 */
Email extractMax(MaxHeap *hp)
{
    Email maxEmail = hp->heap[0];

    /*
     * Move the last element to the root.
     */
    hp->size--;

    if (hp->size > 0) {
        hp->heap[0] = hp->heap[hp->size];

        /* Restore max-heap property */
        heapifyDown(hp, 0);
    }

    return maxEmail;
}

```

```

/*
 * Processes an EMAIL command.
 *
 * Expected format:
 *
 * EMAIL Boss, Meeting with investors, 09-15-2026
 */
void processEmail(MaxHeap *hp, char *line)
{
    Email email;

    char *firstComma;
    char *secondComma;

    /*
     * Find the commas separating:
     * category, subject, date
     */
    firstComma = strchr(line, ',');

    if (firstComma == NULL) {

```



```

    fprintf(stderr, "Invalid EMAIL command.\n");
    return;
}

secondComma = strchr(firstComma + 1, ',');

if (secondComma == NULL) {
    fprintf(stderr, "Invalid EMAIL command.\n");
    return;
}

/*
 * Temporarily terminate the strings at the commas.
 */
*firstComma = '\0';
*secondComma = '\0';

/*
 * line now contains:
 *
 * Boss
 *
 * firstComma + 1 contains:
 *
 * Meeting with investors
 *
 * secondComma + 1 contains:
 *
 * 09-15-2026
 */

/* Skip the "EMAIL " portion */
char *sender = line + 6;

/* Remove leading spaces */
while (*sender == ' ')
    sender++;

char *subject = firstComma + 1;

while (*subject == ' ')
    subject++;

char *date = secondComma + 1;

```

```

while (*date == ' ')
    date++;

/*
 * Remove trailing newline from date.
 */
date[strcspn(date, "\r\n")] = '\0';

/*
 * Copy data into the Email structure.
 */
strncpy(email.sender, sender, MAX_SENDER - 1);
email.sender[MAX_SENDER - 1] = '\0';

strncpy(email.subject, subject, MAX_SUBJECT - 1);
email.subject[MAX_SUBJECT - 1] = '\0';

strncpy(email.date, date, MAX_DATE - 1);
email.date[MAX_DATE - 1] = '\0';

/* Calculate priority and date value */
email.priority = getPriority(email.sender);
email.dateValue = parseDate(email.date);

/* Add email to the heap */
insert(heap, email);
}

/*
 * Main program.
 */
int main(void)
{
    MaxHeap hp;

    char line[512];

    /* Initialize the priority queue */
    initHeap(&hp);

    /*
     * Read commands until EOF.
     */

```

```

*/
while (fgets(line, sizeof(line), stdin) != NULL) {

    /*
     * Remove trailing newline.
     */
    line[strcspn(line, "\r\n")] = '\0';

    /*
     * Ignore empty lines.
     */
    if (strlen(line) == 0)
        continue;

    /*
     * EMAIL command
     */
    if (strncmp(line, "EMAIL ", 6) == 0) {
        processEmail(&hp, line);
    }

    /*
     * NEXT command
     *
     * Does NOT remove the email.
     */
    else if (strcmp(line, "NEXT") == 0) {

        if (hp.size == 0) {
            printf("No emails to read.\n");
        } else {
            Email next = peek(&hp);

            printf("Next email:\n");
            printf("Sender: %s\n", next.sender);
            printf("Subject: %s\n", next.subject);
            printf("Date: %s\n", next.date);
        }
    }

    /*
     * READ command
     *
     * Removes the highest-priority email but does not

```

```

    * print its details.
    */
    else if (strcmp(line, "READ") == 0) {

        if (hp.size > 0) {
            extractMax(&hp);
        }
    }

    /*
    * COUNT command
    */
    else if (strcmp(line, "COUNT") == 0) {
        printf("There are %d emails to read.\n", hp.size);
    }

    /*
    * Unknown command
    */
    else {
        fprintf(stderr, "Unknown command: %s\n", line);
    }
}

/*
* Free the dynamically allocated heap array.
*/
free(hp.heap);

return 0;
}

```

Claude's Code:

```

/* =====
* main.c - CEO Email Priority System
*
* Implements a MaxHeap-based priority queue for emails, built
* entirely from scratch (no library heap/PQ types are used).
*
* Priority rules:
* 1. Sender category (primary key) - higher category wins:

```

```

*   Boss          = 5
*   Subordinate    = 4
*   Peer           = 3
*   ImportantPerson = 2
*   OtherPerson    = 1
*   2. Date (secondary key) - if categories tie, the NEWER
*   email (later date) wins.
*
* Commands (read line-by-line from stdin):
*   EMAIL <category>, <subject>, <MM-DD-YYYY>
*   NEXT
*   READ
*   COUNT
*   ===== */

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

/* -----
* Config constants
* ----- */
#define INITIAL_CAPACITY 8 /* starting size of the heap array */
#define MAX_LINE_LEN 512 /* max length of one input line */
#define MAX_SENDER_LEN 64 /* max length of category/sender str */
#define MAX_SUBJECT_LEN 256 /* max length of subject line */
#define MAX_DATE_LEN 11 /* "MM-DD-YYYY" + null terminator */

```

```

/* -----
* Email data structure
*
* NOTE: The EMAIL command only supplies a sender *category*
* (e.g. "Boss"), not an individual person's name, so we store
* that category string in the `sender` field for display
* purposes (it doubles as the "who this is from" label).
* `priority` is the numeric rank derived from that category and
* is what the comparator actually uses to order the heap.
* ----- */

```

```

typedef struct {
    char sender[MAX_SENDER_LEN]; /* sender category, e.g. "Boss" */
    char subject[MAX_SUBJECT_LEN]; /* subject line */
    char dateStr[MAX_DATE_LEN]; /* original date text, for print*/
    int month, day, year; /* parsed date, for comparison */
    int priority; /* numeric priority (1-5) */
}

```

```

} Email;

/* -----
 * MaxHeap data structure - a dynamically-resizing array of
 * Email structs plus bookkeeping for current size/capacity.
 * ----- */
typedef struct {
    Email *heap; /* dynamically allocated array of Emails */
    int size; /* number of elements currently stored */
    int capacity; /* allocated capacity of the array */
} MaxHeap;

/* =====
 * compare()
 *
 * Standard three-way comparator: returns
 * 1 if a should be considered "greater than" b
 * -1 if a should be considered "less than" b
 * 0 if they are equal in priority
 *
 * Primary key: category priority (bigger number = more urgent).
 * Secondary key (tie-break): date, newest wins.
 * ===== */
int compare(Email a, Email b) {
    if (a.priority != b.priority) {
        return (a.priority > b.priority) ? 1 : -1;
    }

    /* Same category -> compare dates, newest (larger) wins */
    if (a.year != b.year) {
        return (a.year > b.year) ? 1 : -1;
    }
    if (a.month != b.month) {
        return (a.month > b.month) ? 1 : -1;
    }
    if (a.day != b.day) {
        return (a.day > b.day) ? 1 : -1;
    }

    return 0; /* fully equal */
}

/* =====
 * Small helpers

```

```

* ===== */

/* Trims leading/trailing whitespace from a string, in place. */
static void trim(char *s) {
    /* trim leading whitespace by shifting the string left */
    int start = 0;
    while (s[start] == ' ' || s[start] == '\t') start++;
    if (start > 0) {
        memmove(s, s + start, strlen(s + start) + 1);
    }

    /* trim trailing whitespace / newline characters */
    int len = (int)strlen(s);
    while (len > 0 &&
        (s[len - 1] == ' ' || s[len - 1] == '\t' ||
         s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[len - 1] = '\0';
        len--;
    }
}

/* Maps a sender-category string to its numeric priority.
 * Returns 0 (invalid) if the category is unrecognized. */
static int categoryToPriority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0; /* unknown category */
}

/* =====
 * MaxHeap core operations
 * ===== */

/* Allocates the backing array and resets size/capacity. */
void initHeap(MaxHeap *hp) {
    hp->heap = (Email *)malloc(sizeof(Email) * INITIAL_CAPACITY);
    if (hp->heap == NULL) {
        fprintf(stderr, "Fatal error: could not allocate heap memory.\n");
        exit(EXIT_FAILURE);
    }
    hp->size = 0;
}

```

```

    hp->capacity = INITIAL_CAPACITY;
}

/* Doubles the heap's array capacity when it's full. */
static void resizeHeap(MaxHeap *hp) {
    int newCapacity = hp->capacity * 2;
    Email *newBlock = (Email *)realloc(hp->heap, sizeof(Email) * newCapacity);
    if (newBlock == NULL) {
        fprintf(stderr, "Fatal error: could not resize heap memory.\n");
        exit(EXIT_FAILURE);
    }
    hp->heap = newBlock;
    hp->capacity = newCapacity;
}

/* Swaps two Email structs by value. */
static void swapEmail(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* Restores the heap property by moving the element at `index`
 * UP toward the root as long as it's "greater than" its parent.
 * Used right after inserting a new element at the end. */
static void heapifyUp(MaxHeap *hp, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare(hp->heap[index], hp->heap[parent]) > 0) {
            swapEmail(&hp->heap[index], &hp->heap[parent]);
            index = parent;
        } else {
            break; /* parent already outranks (or ties) child; done */
        }
    }
}

/* Restores the heap property by moving the element at `index`
 * DOWN toward the leaves, always swapping with the larger child.
 * Used after removing the root (extractMax). */
static void heapifyDown(MaxHeap *hp, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;

```



```

    int largest = index;

    if (left < hp->size && compare(hp->heap[left], hp->heap[largest]) > 0) {
        largest = left;
    }
    if (right < hp->size && compare(hp->heap[right], hp->heap[largest]) > 0) {
        largest = right;
    }

    if (largest == index) {
        break; /* subtree already satisfies heap property */
    }

    swapEmail(&hp->heap[index], &hp->heap[largest]);
    index = largest;
}

/* Inserts a new Email into the heap, growing the array if needed. */
void insert(MaxHeap *hp, Email email) {
    if (hp->size == hp->capacity) {
        resizeHeap(hp);
    }
    hp->heap[hp->size] = email;
    heapifyUp(hp, hp->size);
    hp->size++;
}

/* Returns the root element WITHOUT removing it. Caller must check
 * hp->size > 0 first (this function is only called after that
 * check in main(), but we guard here too for safety). */
Email peek(MaxHeap *hp) {
    if (hp->size == 0) {
        /* Return a harmless empty Email; caller should never use
         * this because it should have checked size beforehand. */
        Email empty = {"", "", "", 0, 0, 0, 0};
        return empty;
    }
    return hp->heap[0];
}

/* Removes and returns the highest-priority Email, re-heapifying
 * the remaining elements so the heap property still holds. */
Email extractMax(MaxHeap *hp) {

```

```

Email top = peek(heap); /* safe even on empty heap (returns dummy) */

if (hp->size == 0) {
    return top;
}

/* Move the last element to the root, shrink size, sift down. */
hp->heap[0] = hp->heap[hp->size - 1];
hp->size--;
heapifyDown(hp, 0);

return top;
}

/* Frees the heap's backing array. Call once at program exit. */
void freeHeap(MaxHeap *hp) {
    free(hp->heap);
    hp->heap = NULL;
    hp->size = 0;
    hp->capacity = 0;
}

/* =====
 * Command parsing / handling
 * ===== */

/* Parses "MM-DD-YYYY" into three ints. Returns 1 on success,
 * 0 if the string isn't in the expected format. */
static int parseDate(const char *dateStr, int *month, int *day, int *year) {
    /* sscanf with a strict format string; must consume exactly
     * 3 integers separated by hyphens. */
    if (sscanf(dateStr, "%d-%d-%d", month, day, year) != 3) {
        return 0;
    }
    return 1;
}

/* Handles a single "EMAIL <category>, <subject>, <date>" line.
 * `args` is everything after the "EMAIL " keyword. */
static void handleEmailCommand(MaxHeap *hp, char *args) {
    char category[MAX_SENDER_LEN];
    char subject[MAX_SUBJECT_LEN];
    char dateStr[MAX_DATE_LEN];

```

```

/* Split on commas: category , subject , date */
char *part1 = strtok(args, ",");
char *part2 = strtok(NULL, ",");
char *part3 = strtok(NULL, ",");

if (part1 == NULL || part2 == NULL || part3 == NULL) {
    printf("Error: malformed EMAIL command. Expected format:\n");
    printf(" EMAIL <category>, <subject>, <MM-DD-YYYY>\n");
    return;
}

trim(part1);
trim(part2);
trim(part3);

/* Defensive copies into fixed-size buffers */
strncpy(category, part1, MAX_SENDER_LEN - 1);
category[MAX_SENDER_LEN - 1] = '\0';
strncpy(subject, part2, MAX_SUBJECT_LEN - 1);
subject[MAX_SUBJECT_LEN - 1] = '\0';
strncpy(dateStr, part3, MAX_DATE_LEN - 1);
dateStr[MAX_DATE_LEN - 1] = '\0';

int priority = categoryToPriority(category);
if (priority == 0) {
    printf("Error: unknown sender category '%s'. Valid categories are:\n", category);
    printf(" Boss, Subordinate, Peer, ImportantPerson, OtherPerson\n");
    return;
}

int month, day, year;
if (!parseDate(dateStr, &month, &day, &year)) {
    printf("Error: could not parse date '%s'. Expected MM-DD-YYYY.\n", dateStr);
    return;
}

Email e;
strncpy(e.sender, category, MAX_SENDER_LEN - 1);
e.sender[MAX_SENDER_LEN - 1] = '\0';
strncpy(e.subject, subject, MAX_SUBJECT_LEN - 1);
e.subject[MAX_SUBJECT_LEN - 1] = '\0';
strncpy(e.dateStr, dateStr, MAX_DATE_LEN - 1);
e.dateStr[MAX_DATE_LEN - 1] = '\0';
e.month = month;

```

```

    e.day = day;
    e.year = year;
    e.priority = priority;

    insert(hp, e);
}

/* Handles "NEXT": show the top email without removing it. */
static void handleNextCommand(MaxHeap *hp) {
    if (hp->size == 0) {
        printf("No emails in the queue.\n");
        return;
    }
    Email top = peek(hp);
    printf("Next email:\n");
    printf("Sender: %s\n", top.sender);
    printf("Subject: %s\n", top.subject);
    printf("Date: %s\n", top.dateStr);
}

/* Handles "READ": pop the top email, print nothing about it. */
static void handleReadCommand(MaxHeap *hp) {
    if (hp->size == 0) {
        printf("No emails to read.\n");
        return;
    }
    extractMax(hp); /* discard the returned Email; no output */
}

/* Handles "COUNT": report how many emails remain unread. */
static void handleCountCommand(MaxHeap *hp) {
    printf("There are %d emails to read.\n", hp->size);
}

/* =====
 * main() - reads commands line-by-line from stdin until EOF.
 * ===== */
int main(void) {
    MaxHeap hp;
    initHeap(&hp);

    char line[MAX_LINE_LEN];

    while (fgets(line, sizeof(line), stdin) != NULL) {

```

```

/* Strip trailing newline before processing */
size_t len = strlen(line);
if (len > 0 && line[len - 1] == '\n') {
    line[len - 1] = '\0';
}

/* Skip blank lines */
char trimmedLine[MAX_LINE_LEN];
strncpy(trimmedLine, line, MAX_LINE_LEN - 1);
trimmedLine[MAX_LINE_LEN - 1] = '\0';
trim(trimmedLine);
if (strlen(trimmedLine) == 0) {
    continue;
}

if (strncmp(trimmedLine, "EMAIL ", 6) == 0) {
    /* Pass everything after "EMAIL " to the handler */
    handleEmailCommand(&hp, trimmedLine + 6);
} else if (strcmp(trimmedLine, "NEXT") == 0) {
    handleNextCommand(&hp);
} else if (strcmp(trimmedLine, "READ") == 0) {
    handleReadCommand(&hp);
} else if (strcmp(trimmedLine, "COUNT") == 0) {
    handleCountCommand(&hp);
} else {
    printf("Unknown command: %s\n", trimmedLine);
}
}

freeHeap(&hp);
return 0;
}

```

Code Analysis:

Guided by Google Gemini

	Claude	ChatGPT
Correctness	High. Correctly handles missing tokens, edge case trimming of extra spaces/newlines, and safe string bounds Includes freeHeap() to clean memory at runtime exit.	Moderate-High. Performs correctly under standard input. However, fails to free memory allocated in initHeap if memory allocation fails inside insert()
Execution Time	$O(\log N)$ Slightly slower overall because every heap reorganization performs up to 3 if checks to compare dates, adding small CPU branch evaluation costs repeatedly during sorting.	$O(\log N)$ Slightly faster overall because parsing does the math upfront once during insertion, allowing heap re-ordering to run single fast integer comparisons without extra conditional checks.
Space Complexity	$O(N)$ auxiliary space for dynamic array of capacity N Fixed buffer per email struct: $64 + 256 + 11 + (4 \times \text{sizeof}(\text{int}))$ is about 347 bytes per element.	$O(N)$ auxiliary space for fixed buffer per email struct: $50 + 200 + 11 + (2 \times \text{sizeof}(\text{int}))$ is about 269 bytes per element.
Maintainability	High. Functions are well-scoped with explicit comments, defined constant limits (MAX_SUBJECT_LEN), separate memory cleanup functions, and clear modular structure.	Moderate. Clean readable standard C style, but lacks explicit memory cleanup functions (freeHeap), leaving dynamic array deallocation inline at the end of main(). Inline string manipulation in processEmail is slightly harder to follow.

f. The analysis given by Gemini has led me to pick Claude's code as the better option. Claude has better maintainability and correctness, preventing errors in input and memory management. ChatGPT had slightly better space and time but it was negligent when it came to deciding which code to pick because the code from Claude was much more ahead in the other categories.

G.

1. Correctness: Improved the `parseDate()` function to make sure the input date is valid and not a date like 2-31-2026 for example
2. Execution Time: Took ChatGPT's YYMMDD integer structure for dates to switch one comparison instead of 3 within the `compare()` function to save some time
3. Space: Gemini was able to find string copies within struct members where numerical representations suffice and remove them from the code.
4. Maintainability: Gemini extracted string-parsing logic into isolated pure functions (`parse_email_line()`, `heap_destroy()`) to keep `main()` concise and focused strictly on the command-dispatch loop.
 - a. Also added clear comments on every line and prologue comments citing the sources and explaining all of it.